Claude Code for Empirical Economists: A Reference Compendium

Sankalp Sharma
Department of Economics, University of Southern California
ssharma9@usc.edu

April 27, 2026

Abstract

Claude Code is Anthropic's agentic command-line interface to its frontier large language models. Unlike chatbots or in-editor autocompleters, it runs a stateful read–plan–edit–run–verify loop with full filesystem and shell access, acting as a research assistant across an entire empirical project: cleaning data, estimating models, building replication packages, drafting manuscripts, and auditing the result. This compendium covers four areas: the technical fundamentals of the tool as of v2.1; the economics-specific ecosystem that emerged in 2025–26 (Goldsmith-Pinkham, Cunningham, Sant'Anna, Panjwani, Korinek, Grundl); concrete workflows for Stata, R, Python, and LaTeX; and recommended configurations for safety, cost, reproducibility, and AEA-compliant replication. On standardized researcher-variation tasks, Claude Code matches trained human researchers in central tendency but cuts across-run dispersion by a factor of two to twenty-seven (Grundl, 2026). Three risks dominate for empirical economists: Stata-specific API hallucination, methodological drift on novel extensions, and unsafe handling of restricted data. Each is catalogued below with a prescribed mitigation.

Contents

1 Introduction **3**

2 Technical Fundamentals **3**
2.1 What Claude Code Is . . . 3
2.2 Installation and Authentication . . . 3
2.3 Built-in Tools . . . 4
2.4 The CLAUDE.md Memory Hierarchy . . . 4
2.5 Slash Commands . . . 5
2.6 Custom Commands and Skills . . . 5
2.7 Subagents and the Agent Tool . . . 5
2.8 Model Context Protocol (MCP) . . . 6
2.9 Permission Modes . . . 6
2.10 Context Management . . . 7
2.11 Hooks . . . 7
2.12 Models and Costs . . . 7
2.13 Headless Mode and CI . . . 8
2.14 Recent Changes (v2.1 era) . . . 8

3 Annotated Literature: Economics-Specific Resources **8**
3.1 Tier 1: Essential References . . . 8
3.2 Tier 2: Very Useful . . . 10
3.3 Tier 3: Adjacent and Skeptic Counterweights . . . 13

4 Applied Workflows by Language and Task **13**
4.1 Data Cleaning and Wrangling . . . 13
4.2 Econometric Analysis . . . 13
4.3 Replication Packages (AEA) . . . 14
4.4 LaTeX . . . 14
4.5 Literature Review and PDF Handling . . . 15
4.6 HPC and SLURM . . . 15
4.7 GIS and Spatial Workflows . . . 15
4.8 Polyglot Projects (Stata + R + Python + LaTeX) . . . 16

5 Recommended Practices **16**
5.1 A Composite `CLAUDE.md` Template . . . 16
5.2 A Recommended `settings.json` for Academics . . . 17
5.3 Git Workflow and Worktrees . . . 17
5.4 Sensitive Data: What Not to Send . . . 17
5.5 Cost Management . . . 18
5.6 Common Pitfalls . . . 19
5.7 Replication Audits . . . 19
5.8 Prompt Engineering for Research . . . 19

6 Empirical Evidence on Capability and Failure Modes **20**
6.1 Grundl (2026): The Researcher-Variation Benchmark . . . 20
6.2 Straus and Hall (2026): The Replication Audit . . . 20
6.3 Xu and Yang (2026): Reproducibility at Scale . . . 20
6.4 Synthesis . . . 20

7 Conclusion **21**

Reading List **21**

1 Introduction

Frontier large language models have entered empirical economic research in earnest. Anthropic's *Claude Code* — an agentic terminal CLI launched in early 2025, now shipping as v2.1 — breaks from the chat assistants (ChatGPT, Claude.ai) and in-editor autocompleters (Copilot, Cursor) that economists have used since 2023. The defining property is *agency*: given a research goal, Claude Code reads files, plans, edits source across many files, runs shell commands, inspects logs, and iterates. Lead engineer Boris Cherny calls it "a Unix utility, not an IDE."

Three properties make the shift consequential for working economists. First, the loop grounds output in execution: code that fails to compile or to produce expected numbers is caught at runtime. The hallucinations that plague retrieval-style LLM use — invented citations, fabricated coefficients — collapse when the model must run what it writes. Second, the marginal cost of an additional specification approaches zero. Anthropic reports active-day spending of roughly $6 per user, with 90% of users below $12. Third, a credible benchmarking literature now exists. Grundl (2026) reruns the Huntington-Klein–Pörtner researcher-variation experiment with Claude Code playing the "researcher" and finds that estimated treatment effects match human researchers in mean and median while exhibiting two-to-twenty-seven times lower across-run dispersion.

The economist-specific literature matured rapidly in late 2025 and early 2026. Substantial public guides now come from Goldsmith-Pinkham (2026), Cunningham (2025), Sant'Anna (2026), Panjwani (2026), and Korinek (2025). This compendium consolidates rather than duplicates that material, organizing it into a single reference for the working empirical economist.

The paper proceeds as follows. Section 2 catalogues technical fundamentals. Section 3 surveys the economist-specific resources. Section 4 documents workflows by language and task. Section 5 prescribes configurations and operational practices. Section 6 reviews the empirical evidence on capability and failure. Section 7 concludes.

2 Technical Fundamentals

2.1 What Claude Code Is

Claude Code is a command-line program that drives Anthropic's Claude family of models (currently Opus 4.7, Sonnet 4.6, and Haiku 4.5). The CLI exposes the model to the local filesystem and shell, equips it with a fixed toolkit — file read/write, search, bash, web fetch, sub-agent spawning — and runs it inside a perception–action loop. The same engine powers Claude.ai, the VS Code and JetBrains plugins, the desktop and iOS apps, `claude.ai/code`, and the Claude Agent SDK (Python, TypeScript). This document treats the CLI, the most flexible of these surfaces.

2.2 Installation and Authentication

The recommended installer (2026) is the Anthropic-hosted bash script:

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash
# Windows PowerShell
irm https://claude.ai/install.ps1 | iex
```

Alternatives include Homebrew (`brew install -cask claude-code`), winget (`winget install Anthropic.ClaudeCode`), and npm (`npm install -g @anthropic-ai/claude-code`, requires Node

18+).

Authentication routes:

- Browser OAuth using a Pro, Max, Team, or Enterprise account on claude.ai. The free tier is *not* eligible.
- `ANTHROPIC_API_KEY` environment variable for pay-as-you-go API billing.
- Amazon Bedrock (`CLAUDE_CODE_USE_BEDROCK=1`), Google Vertex AI, Microsoft Foundry, or LiteLLM proxy for institutional cloud routing.

Verification: `claude -version` confirms install; `claude doctor` diagnoses install, authentication, and configuration. Launch modes: `claude` (interactive REPL), `claude "fix bug X"` (interactive with seed prompt), `claude -p "..."` (headless, single-shot), `claude -c` or `claude -resume` (continue a prior session).

2.3 Built-in Tools

The model has access to a fixed toolkit that it invokes by name during its agentic loop. Core tools: `Read`, `Write`, `Edit`, `MultiEdit`, `NotebookEdit`, `Bash`, `Glob`, `Grep`, `WebFetch`, `WebSearch`, `Agent` (renamed from `Task` in v2.1.63), `TodoWrite`, `AskUserQuestion`, `ExitPlanMode`, and `EnterWorktree`. Anthropic deliberately chose agentic search via grep and glob over retrieval-augmented generation (RAG) with vector stores, on the basis of internal benchmarks reported by Cat Wu and Boris Cherny.

2.4 The CLAUDE.md Memory Hierarchy

`CLAUDE.md` files are plain-Markdown documents loaded into every session as part of the system prompt. The hierarchy is:

Scope	**Path**	**Shared via**
Managed policy	`/etc/claude-code/CLAUDE.md` (Linux)	Organization
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Git
User	`~/.claude/CLAUDE.md`	Personal, all projects
Local	`./CLAUDE.local.md` (gitignored)	Personal, this project

Table 1: The CLAUDE.md memory hierarchy. More specific scopes override broader ones.

The CLI walks up the directory tree concatenating all encountered `CLAUDE.md` files. Subdirectory `CLAUDE.md` files load on demand when the model accesses files in that subtree. Imports are supported via `@README.md`, `@docs/conventions.md`, `@~/.claude/personal-rules.md` syntax to a maximum depth of five.

A new *auto-memory* facility (v2.1.59+) writes to `~/.claude/projects/<proj>/memory/MEMORY.md` as Claude takes its own notes; the first 200 lines or 25 KB are loaded into each subsequent session. The project-root `CLAUDE.md` is preserved through `/compact`.

The canonical length guidance from both Anthropic and HumanLayer is to keep the project `CLAUDE.md` under 200 lines. HumanLayer's own root file is under 60. Each line should pass the test "would removing this cause Claude to make mistakes?" If not, cut it. The directives `IMPORTANT` and `YOU MUST` materially increase adherence but are diluted by overuse.

2.5 Slash Commands

Built-in slash commands as of v2.1.120: `/help`, `/init`, `/clear`, `/compact [instructions]`, `/usage` (merged from `/cost` and `/stats`), `/model`, `/effort low|medium|high|max`, `/agents`, `/mcp`, `/permissions`, `/hooks`, `/memory`, `/review`, `/security-review`, `/pr_comments`, `/resume`, `/rewind` (or `Esc Esc`), `/rename`, `/export`, `/config`, `/vim`, `/bug`, `/doctor`, `/status`, `/context`, `/plan`, `/btw` (sidebar question; does not enter context), `/feedback`, `/desktop`, `/teleport`, `/statusline`, `/theme`, `/plugin`, `/skills`, `/loop`, `/schedule`. Bundled higher-level skills include `/simplify`, `/batch`, `/debug`, `/code-review`, `/ultrareview`, and `/ultraplan`.

2.6 Custom Commands and Skills

In the 2026 architecture, custom commands have been merged with skills. A file at `.claude/commands/deploy.md` and one at `.claude/skills/deploy/SKILL.md` both expose `/deploy`. The recommended format uses YAML frontmatter:

```
---
name: regression-table
description: Convert estout output to publication LaTeX table
allowed-tools: Bash(esttab *) Read Edit
model: sonnet
paths: ["code/analysis/**"]
argument-hint: "[spec-name]"
---
Generate a booktabs LaTeX table from $ARGUMENTS using esttab conventions...
```

Available variables: `$ARGUMENTS`, `$1`, `${CLAUDE_SESSION_ID}`, `${CLAUDE_PLUGIN_ROOT}`. Dynamic context can be injected with the `!` prefix (e.g., a line beginning "`- PR diff: ¡gh pr diff‘`" embeds live shell output).

2.7 Subagents and the Agent Tool

Subagents run in a separate context window with their own system prompt and tool restrictions, returning only a summary to the main thread. Built-in subagents include `Explore` (read-only search), `Plan` (research during plan mode), `general-purpose`, `Bash`, and `claude-code-guide`. Custom subagents are defined in `.claude/agents/<name>.md`:

```
---
name: referee-2
description: Empirical-economics critic. Use proactively after analysis changes.
tools: Read, Grep, Bash(Rscript *), Bash(stata -b *)
model: opus
---
You are Referee 2 for AER. Adversarially critique identification, SE clustering, robustness...
```

Invocation routes: automatic (the main agent reads the description and decides), explicit natural language ("use the referee-2 subagent to..."), `@referee-2` mention, or session-wide via `claude -agent referee-2`. Up to approximately seven parallel subagents have been observed in practice. Subagents *cannot* spawn further subagents, which prevents recursive nesting.

The critical pattern for economists: subagents preserve main-thread context. A *Referee 2* subagent that reads 200 files and returns a one-paragraph critique consumes those tokens in its own window; the main session sees only the summary.

2.8 Model Context Protocol (MCP)

MCP is an open standard for connecting Claude Code to external tools. Servers are added via:

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
    --header "Authorization: Bearer $GH_PAT"
claude mcp add --transport stdio --env STATA_PATH=/usr/local/stata18 \
    stata -- npx -y stata-mcp
```

Scopes: User (`~/.claude.json`), Project (`.mcp.json`, committed to the repo), and Local (default, machine-specific).

Servers of particular interest to economists:

- **stata-mcp** (`sepinetam`; also `hanlulong`) — drives Stata directly, enabling iteration on errors.
- **playwright** / **puppeteer** — browser automation for portal scraping.
- **filesystem**, **github**, **postgres**, **dbhub** — data and version-control plumbing.
- **context7** — live, current package documentation.
- **sequential-thinking** — structured chain-of-thought scaffolding.
- **slurm-mcp** (`dongwookim-ml`) — remote HPC job submission.
- **fred-mcp**, **bls-mcp**, **census-mcp** — direct access to U.S. statistical agencies.

Tool Search is the default discovery mechanism in 2026: rather than loading every MCP tool's schema into every prompt, MCP tool schemas are deferred and fetched on demand. This is governed by `ENABLE_TOOL_SEARCH=true|auto|auto:N`.

2.9 Permission Modes

`Shift+Tab` cycles through modes:

Mode	Behavior
`default`	Prompt before edits, Bash, MCP
`acceptEdits`	Auto-approve edits; prompt for Bash
`plan`	Read-only; no changes permitted
`auto`	Background classifier model approves safe operations
`dontAsk`	Pre-approved tools only
`bypassPermissions`	All checks skipped (`-dangerously-skip-permissions`); hooks still fire

Table 2: Permission modes.

Per-tool allow and deny rules are configured in `.claude/settings.json`:

```
{"permissions": {
  "allow": ["Read","Glob","Grep","Bash(git status:*)","Bash(Rscript:*)"],
  "deny": ["Read(./.env*)","Read(./data/raw/**)","Read(./irb/**)",
           "Bash(rm -rf:*)","Bash(git push --force:*)"]
}}
```

Important caveat (January 2026, multiple GitHub issues): deny rules are silently bypassed for compound bash commands and absolute paths. Hard enforcement requires `PreToolUse` hooks (Section 2.11).

2.10 Context Management

The standard context window is 200,000 tokens; Opus 4.7 / 4.6 and Sonnet 4.6 support a one-million-token window at standard pricing. Auto-compaction triggers at approximately 75–80% utilization. Tools for context discipline:

- `/clear` — full reset; irreversible.
- `/compact [instructions]` — summarize current context, retaining instructed elements.
- `Esc Esc` or `/rewind` — checkpoint menu; revert to a prior turn.
- `/btw` — ask an out-of-band question without polluting the working context.
- `/context` — visualize current token allocation.

The dominant anti-pattern is the *kitchen-sink session*: a single REPL accumulating dozens of unrelated tasks. Anthropic's recommended discipline is to `/clear` between unrelated tasks, and after two consecutive failed corrections, to `/clear` and rewrite the prompt with the lessons learned.

2.11 Hooks

Hooks are deterministic shell or HTTP commands that fire at lifecycle events. The aphorism is: *hooks guarantee behavior; prompts suggest it.* Events include `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `PreCompact`, `PostCompact`, and `Stop`. The exit codes that matter: `0` proceed, `2` block, `1` warn-only.

Common patterns: auto-format on Edit, block `rm -rf` or `git push -force`, inject git status at `SessionStart`, re-inject working context after `PostCompact`, block edits to `.env` or `raw_data/`. Hooks fire even when `-dangerously-skip-permissions` is set — they are the safety net of last resort.

2.12 Models and Costs

Model	Alias	Input/Output (USD/M)	Use for
Opus 4.7	`opus`	\$5 / \$25	Architecture, hard bugs, replication audits
Opus 4.6	(legacy)	\$5 / \$25	Same; older tokenizer
Sonnet 4.6	`sonnet`	\$3 / \$15	Daily driver
Sonnet 4.5	(legacy)	\$3 / \$15	ZDR-eligible workloads
Haiku 4.5	`haiku`	\$1 / \$5	Bulk mechanical work

Table 3: Pricing as of April 2026. Opus 4.7 produces approximately 35% more tokens via a new tokenizer.

Prompt caching: cache writes are billed at 1.25× (5-minute TTL) or 2× (1-hour TTL); cache reads are billed at 0.1× — a 90% discount. The special alias `-model opusplan` runs Opus during planning and Sonnet during execution. The flag `-max-budget-usd 5.00` hard-caps headless runs. Per-day spend is tracked via `/usage` or via the third-party `npx ccusage daily`, which parses `~/.claude/projects/*.jsonl`.

For academics, the practical recommendation is the Max 5x plan at \$100/month, which is approximately 2–2.5× cheaper than direct API access at typical PhD-student workloads. Pro at \$20/month suffices for one to two short sessions per day. Max 20x at \$200/month and direct API access are reserved for CI, headless batch, and per-project cost attribution. Anthropic's published averages: approximately \$6 per active day, with 90% of users below \$12 per day.

2.13 Headless Mode and CI

```
claude -p "Explain this project" --output-format json
git diff main --name-only | claude -p "Review for SE clustering errors"
claude --permission-mode auto -p "fix all lint errors" --max-budget-usd 2.00
for f in $(cat files.txt); do
  claude -p "Migrate $f to fixest" --allowedTools "Edit"
done
```

The official GitHub Actions integration is `anthropics/claude-code-action`. Bare mode (`-bare`) skips MCP discovery for fast startup.

2.14 Recent Changes (v2.1 era)

Opus 4.7 ships with a new `xhigh` default reasoning effort. `/cost` and `/stats` have been merged into `/usage`. Custom commands have been folded into Skills. Auto-memory and Tool Search are default. The `Task` tool was renamed `Agent`. New facilities include Agent Teams (multi-session coordination), Routines (cloud-scheduled agents), Computer Use preview, voice dictation, and a PowerShell-native tool. The `ultrathink` keyword was deprecated and then re-introduced in v2.1.68; extended thinking is now on by default and is governed by `/effort`.

Canonical primary sources:

- https://code.claude.com/docs/en/best-practices
- https://code.claude.com/docs/en/memory
- https://code.claude.com/docs/en/sub-agents
- https://code.claude.com/docs/en/mcp
- https://code.claude.com/docs/en/hooks-guide
- Anthropic, "How Anthropic teams use Claude Code"

3 Annotated Literature: Economics-Specific Resources

3.1 Tier 1: Essential References

Goldsmith-Pinkham (2026). Yale SOM finance / NBER. The seven-part Markus' Academy mini-series "Claude Code for Applied Economists" (Episodes 162-1 through 162-7, recorded at the Princeton Bendheim Center, March–April 2026) is the most actively viewed long-form reference. Series hub: https://markusacademy.substack.com/p/claude-code-for-applied-economists. Episodes cover Getting Started (https://www.youtube.com/watch?v=HzgByl5ZsWE), Data Collection, Web Scraping, Large Datasets (https://www.youtube.com/watch?v=4uwI1-9DafU), Writing & Thinking (https://www.youtube.com/watch?v=BxfSiB3Moyo), and two further episodes; per-episode write-ups appear at the Markus Academy Substack. Demonstrations include: a **Census home-ownership pipeline** where Claude locates the data, scrapes, cleans, and produces plotting scripts; a **SEC EDGAR 10-K** extraction that builds a DuckDB database of risk-factor disclosures and shows tariff-related risk language rising in frequency and specificity; and an **HMDA mortgage panel** that converts large CSVs to Parquet, indexes them in DuckDB, and produces a county-level lender-concentration panel. The *Writing & Thinking* episode demonstrates building a personal style guide from one's own past papers and using Skills to scaffold a referee-response revision plan. Tooling recommendations: Max plan at $100/month, DuckDB and Parquet for everything above ∼5 GB, and the Skills feature for any reusable workflow.

Cunningham (2025–26). Baylor; author of *Causal Inference: The Mixtape*. Active Substack series at https://causalinf.substack.com/archive (40+ posts as of April 2026) and the `MixtapeTools` repository: https://github.com/scunning1975/MixtapeTools. The repository ships six skills: *Referee 2*, *Blindspot*, *Split-PDF*, *Beautiful Deck*, *TikZ*, and *Newproject*. The **Referee 2** skill is the most consequential single artifact in the economist-facing Claude Code ecosystem: a five-audit protocol — Code Audit, Cross-Language Replication, Directory Audit, Output Automation Audit, Econometrics Audit — run by a fresh Claude instance in a fresh terminal, with no prior exposure to the project. Cunningham himself describes it as "a health inspector for empirical research," delivered as a formal referee report with a revise-and-resubmit pass.

The most viral economist-friendly Claude Code anecdote is Cunningham's replication of Card et al. (2022) (PNAS), which classifies ∼200,000 congressional and 5,000 presidential immigration-related communications. Substituting GPT-4o-mini for the original RoBERTa classifier, Cunningham reports a total cost of **$10.99** and a wall-clock time of ∼**4.6 hours** (1 h setup + 2.6 h compute + 1 h analysis), with **69% agreement** against the original published classifications. The original team's labeling step was estimated at 7 RAs and well over $10,000. See *Claude Code 15: The Results Are In* (https://causalinf.substack.com/p/claude-code-15-the-results-are-in) for the full write-up.

Panjwani (2026). Northwestern economics PhD, ex-Zelle/Early Warning ML engineer; runs *The AI MBA* (https://ai-mba.io). The most polished public Claude-Code-with-Stata tutorial. The YouTube walkthrough "How to Use Claude Code with Stata — PhD Economist Explains" (https://www.youtube.com/watch?v=YLQvQ2MD2Y8) demonstrates the side-by-side terminal/Stata pattern, prompt construction for cleaning and reshaping, and use of `pdfgrep`/`pdfplumber` to query Stata's PDF documentation. The VoxDev/CEPR webinar "AI Agents for Economic Research" (March 2026, ∼750 live attendees), slides at https://voxdev.org/sites/default/files/2026-03/AI_Agents_for_Economic_Research.pdf, generalizes the pattern to research workflows. As of March 2026 Panjwani recommends mixing Codex (80%) with Claude Code (20%) at the higher subscription tiers — a useful signal that the agent-tooling landscape remains in flux.

P. Sant'Anna (2026). Emory; Callaway–Sant'Anna co-author; *Econ 730: Causal Panel Data*. The `claude-code-my-workflow` repository (https://github.com/pedrohcgs/claude-code-my-workflow; guide at https://psantanna.com/claude-code-my-workflow/) is the most fully realized economics-academic Claude Code template. Verified contents: **28 skills, 14 agents, 24 path-scoped rules, and 6 hooks**. Multi-agent adversarial QA is calibrated against AER, QJE, JPE, ECMA, and ReStud expectations. Built during preparation for Econ 730: produced six PhD lecture decks (800+ slides), interactive Quarto versions, and full R replication packages. Adopted by 15+ research groups across economics, energy, political science, and engineering as of March 2026. *Recommended as the "fork tonight if you want depth" starting point.*

H. Sant'Anna (2026). UAB. `Clo-Author` (https://github.com/hugosantanna/clo-author; site at https://hsantanna.org/clo-author/) reorients the Pedro Sant'Anna template from lecture production to a research-paper pipeline. Seventeen specialized agents (including six worker–critic pairs plus referees, a data engineer, and a verifier), 26+ slash commands, 30 journal profiles spanning economics, finance, accounting, and marketing. Phase-based severity: encouraging at *Discovery*, adversarial at simulated *Peer Review*. AEA replication compliance and a blind-review simulation are built in. Validated for DiD, IV, RDD, synthetic control, and event studies.

Korinek (2025). UVA / Brookings / NBER; member of the Anthropic Economic Advisory Council. "AI Agents for Economic Research," NBER Working Paper 34202, **September 2025**: https://www.nber.org/papers/w34202; companion Substack at https://genaiforecon.substack.com/; project hub at GenAIforEcon.org. Korinek (2025) "aims to demystify AI agents — autonomous LLM-based systems that plan, use tools, and execute multi-step research tasks — and provides hands-on instructions for economists to build their own, even without programming expertise." Capabilities demonstrated include autonomous literature reviews across many sources, writing and debugging econometric code, fetching and analyzing economic data, and coordinating multi-step research workflows. The earlier Korinek (2023) (NBER 30957) provides the now widely-cited six-domain taxonomy of LLM use cases for economists — ideation, writing, background research, data analysis, coding, and mathematical derivations — which Korinek (2025) updates and extends to agentic tools. The pair are the authoritative academic citations.

Grundl (2026). Principal Economist, Federal Reserve Board (Research & Statistics). "Claude Code as an Empirical Economist: Like Humans but Without the Tails," SSRN 6219138, posted **February 11, 2026**: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6219138; companion site https://claude-code-economist.com/. The benchmark task is the causal effect of DACA eligibility on full-time employment for Hispanic and Mexican immigrants in the U.S., posed in three task variants. Each AI system runs **100 independent replications** per variant. Means and medians of estimated effects are "fairly similar" between Claude Code and human researchers; across-Claude-run dispersion is "substantial," but human across-researcher dispersion is **two to twenty-seven times larger**. Grundl concludes that AI systems of this class "can serve as scalable tools for producing novel empirical research, large-scale replication, and robustness analysis." This is the first apples-to-apples benchmark of Claude Code against human empirical economists on a real causal-inference task, by a Fed economist — and accordingly the most rhetorically powerful empirical finding available for a skeptical academic audience.

3.2 Tier 2: Very Useful

Blattman (2026). Chris Blattman, Ramalee E. Pearson Professor of Global Conflict Studies, Harris School, University of Chicago. *Claude Blattman*: https://claudeblattman.com/ (site launched January 2026; companion repo at https://github.com/chrisblattman/claudeblattman). Blattman is the most prominent senior, self-described non-coding social scientist using Claude Code as an "executive function" tool. The site documents skills for inbox triage, morning briefings, meeting prep and follow-up, project dashboards, proposal drafting, and trip planning. The most-circulated anecdote: applying a custom Gmail-filter triage skill, his inbox dropped from ∼5,000–5,600 unread emails to six in roughly a day and a half (X thread https://x.com/cblatts/status/2027018464670491065). He maintains weekly project-status skills that ingest meeting transcripts, project emails, and WhatsApp threads for ∼10 active research projects (Executive Assistant page: https://claudeblattman.com/toolkit/executive-assistant/). Blattman: "Four weeks ago I was a Claude Code skeptic. Now I'm 120% sold." The site is the proof-of-concept that the gains do not require coding fluency.

Straus and Hall (2026). Graham Straus (UCLA PhD student) and Andrew B. Hall (Stanford GSB). "How Accurately Did Claude Code Replicate and Extend a Published Political Science Paper?" https://www.andrewbenjaminhall.com/Straus_Hall_Claude_Audit.pdf. The audited paper is Thompson et al. (2020) (PNAS), "Universal vote-by-mail has no impact on partisan turnout or vote share." Documented findings: Claude exactly replicates the original headline estimates; correctly codes **29 of 30 California counties** on treatment timing; and on a new election-data

collection task achieves correlation **above 0.999** with Straus's manual hand collection. Documented error modes on extensions: drift from the prompt, missed races, data collected and never used, poor documentation of analytic choices, and one substantive inconsistency where Claude wrote that it had restricted to eventual VCA-adopting CA counties while the event-study code retained never-treated observations. No outright hallucinations or "crazy errors." A companion thread by Hall (https://x.com/ahall_research/status/2007603340939800664) documents Claude Code writing a complete empirical political-science paper in approximately one hour. This is the cleanest published capability-and-failure demonstration for an empirical social-science audience.

Pepinsky (2026). Tom Pepinsky, Walter F. LaFeber Professor of Government, Cornell. "Agentic AI and Social Science Research Practice," January 23, 2026: https://tompepinsky.com/2026/01/23/agentic-ai-and-social-science-research-practice/. Pepinsky calls agentic AI a "seismic shift" for social science and writes that "Claude Code writes and executes code better and faster than what I can do on my own." His key analytical move is to draw the line between knowledge retrieval (where "the hallucination problem is real" and LLMs cannot reliably produce citations or literature reviews) and code execution (where the agent is grounded). His normative claim — that "the relative value of knowledge and expertise is higher than it was five years ago, and the value of being able to code has plummeted" — is the cleanest framing of the human-capital implication for PhD economists. He cautions that agentic AI is "ripe for p-hacking" because the agent will "produce results it thinks you want to hear," and recommends restricting it to rule-following tasks.

Other key Tier-2 sources.

- **Xu and Yang** (Stanford / HKBU, 2026), "Scaling Reproducibility," https://yiqingxu.org/papers/2026_ai/AI_reproducibility.pdf: 100% reproducibility on 92 papers comprising 215 specifications, in under four minutes per paper. The strongest existence proof for Claude Code as automated AEA Data Editor companion.
- **Cherny and Wu** (Anthropic), Latent Space podcast, May 8, 2025: https://www.latent.space/p/claude-code. Primary source for the design philosophy: early Claude Code prototypes used RAG with vector stores; the team switched to agentic search via grep and glob because it "performs a lot better based on benchmarks and vibes," and avoids the indexing, sync, and external-dependency overheads of RAG. Anthropic-internal stat reported on the episode: **80–90% of Claude Code's own code is written by Claude**; Cherny reports a personal 2× productivity gain.
- **Anthropic Economic Advisory Council** (April 28, 2025): https://www.anthropic.com/news/introducing-the-anthropic-economic-advisory-council. Members: Tyler Cowen (GMU/Mercatus), Oeindrila Dube (Harris), Tomas J. Philipson (Chicago), Silvana Tenreyro (LSE), Chiara Farronato (HBS), John List (Chicago), Ioana Marinescu (Penn), and Anton Korinek (UVA). Charter: advise Anthropic on AI's impact on labor markets, growth, and broader socioeconomic systems; informs the Anthropic Economic Index research agenda.
- **Anthropic** (March 23, 2026), "Long-running Claude for scientific computing": https://www.anthropic.com/research/long-running-Claude. Authored by Siddharth Mishra-Sharma (Anthropic Discovery team). The case study implements a differentiable cosmological Boltzmann solver for the cosmic microwave background — coupled equations across photons, baryons, neutrinos, and dark matter — on an HPC cluster running SLURM (`-partition=GPU-shared`, `-gres=gpu:h100-32:1`, `-time=48:00:00`), with Claude Code inside `tmux` for detach/reattach. The pattern relies on three artifacts: a progress file, a test oracle, and an agent prompt with explicit rules. The *Ralph loop* (named after the community "Ralph Wiggum" pattern, also

shipped as the `ralph-loop` plugin) is defined as "essentially a for loop which kicks the agent back into context when it claims completion, and asks if it's really done," iterating until the success criterion holds. Claim: "can compress months or even years of researcher work into days," though the resulting solver is not production-grade.

- **Stata Blog** (October 7, 2025), "Stata commands to run ChatGPT, Claude, Gemini, and Grok": https://blog.stata.com/2025/10/07/stata-commands-to-run-chatgpt-claude-gemini-and-grok/. First-party StataCorp endorsement of LLM integration into Stata. Updates a 2023 post (the original `chatgpt` ado broke after an OpenAI API change). Provides ado-files for `chatgpt`, `claude`, `gemini`, and `grok` commands; usage e.g. `claude "Write a haiku about Stata."` Maximally legitimizes LLM use for traditional Stata users.
- **Cornwl**, "Claude Code & Cowork for Academic Research," https://cornwl.github.io/files/claude-academic-guide.html: best LaTeX/Beamer/Quarto guide.
- **Hanlu Long**, `awesome-ai-for-economists`, https://github.com/hanlulong/awesome-ai-for-economists curated index. Companion `econ-writing-skill` synthesizes 50+ writing guides (Cochrane, McCloskey, Shapiro).
- **Dylan Moore**, `stata-skill`, https://github.com/dylantmoore/stata-skill: 37 reference files, 20 community packages (`reghdfe`, `ivreg2`, `csdid`, `did_multiplegt`, `rdrobust`, `psmatch2`, `synth`, `binsreg`, `coefplot`, `gtools`).
- **SepineTam**, `stata-mcp`, https://github.com/SepineTam/stata-mcp: MCP server allowing Claude Code to execute Stata. Resolves the "Claude can write `.do` files but cannot iterate on errors" gap.
- **letitbk**, `claude-academic-setup`, https://github.com/letitbk/claude-academic-setup: one-shot installer with 22 skills, 13 plugins, sandbox mode pre-configured.
- **Antonio Mele**, *Awesome Econ AI Stuff*, https://meleantonio.github.io/awesome-econ-ai-stuff/: curated DiD/IV/RD Stata skills.
- **Tyler Ransom** (Associate Professor, Oklahoma), six-month retrospective https://tyleransom.substack.com/p/how-ive-been-using-claude-for-the: concrete Stata, R, and LaTeX prompts. Reports using Claude to write bespoke Stata programs despite 15+ years of Stata use without ever having written his own ado. Verbatim BibTeX-maker prompt: "I'd like you to act as a bibtex entry maker . . . Please include full names as much as possible, put periods after middle initials, and Chicago-style proper capitalize the title. Please include doi (or url if doi is unavailable). If the doi is available don't put in the url. Also don't put the publication month."
- **Aziz Sunderji**, *Home Economics*, "How I use Claude Code": https://homeeconomics.substack.com/p/how-i-use-claude-code: workflow notes from a working economist-turned-data-journalist.
- **Autonomous Econ**, "10 Claude Code Tips for Analysts and Economists": https://autonomousecon.substack.com/p/the-10-claude-code-habits-that-turn: practitioner habits checklist.
- **Mihail Velikov**, AI-Econ Wiki, https://velikov-mihail.github.io/ai-econ-wiki/: episode-by-episode summaries of the Goldsmith-Pinkham series and adjacent talks.
- **Andrew B. Hall**, "The 100x Research Institution": https://freesystems.substack.com/p/the-100x-research-institution: companion essay to the Straus–Hall audit, projecting the institutional consequences.
- **Hanlulong**, `stata-mcp` (distinct from SepineTam's), https://github.com/hanlulong/stata-mcp: Stata Workbench in VS Code with MCP at `localhost:4000/mcp-streamable`.
- **Jill0099**, `causal-inference-mixtape` skill: templates for DiD, RDD, IV, and synthetic control in three languages, derived from Cunningham's *Mixtape*.
- **Z. R. Chen**, workflow site, https://zhiyuanryanchen.github.io/claude-code-workflow.html: Sant'Anna-derived starter.
- **Raul Guarini Riva**, *Resources for Economists*, https://rgriva.github.io/resources: a

model for what an institutional follow-up resource page might look like.

3.3 Tier 3: Adjacent and Skeptic Counterweights

Ethan Mollick (*One Useful Thing*), "Claude Code and What Comes Next"; the Anthropic Skilljar course (anthropic.skilljar.com/claude-code-in-action); the `awesome-claude-code` repositories (hesreallyhim, jqueryscript, VoltAgent); `fcakyon/phd-skills`; `flonat/claude-research`; `Galaxy-Dawn/claude-scholar`; John Horton (MIT Sloan), *Large Language Models as Simulated Economic Agents* and *Automated Social Science: Language Models as Scientist and Subjects* — focused on LLMs as research *subjects* rather than as a coding tool, but adjacent to questions about Claude Code's role in research; Henry Farrell, "AI is great for scientists. Perhaps it's not so great for science" (https://www.programmablemutter.com/p/ai-is-great-for-scientists-perhaps) — the most articulate skeptic counterweight on aggregate-knowledge effects.

4 Applied Workflows by Language and Task

4.1 Data Cleaning and Wrangling

Representative prompts:

1. "Read `raw/cps_2010_2023.dta` (45M rows). Convert to a Polars LazyFrame, build a balanced state-by-year-by-industry panel, create one- and two-year lags of `log_emp` within `state_id` × `naics4`. Write Parquet to `data/derived/`. Verify balance and report missing cells."
2. "Probabilistic record linkage: match `raw/borrowers.csv` (PMJDY beneficiaries) to `raw/sec_credit_bureau.cs` on noisy names plus DOB. Use `splink` with Jaro–Winkler. Output match probabilities and a calibration plot. Mark below 0.85 as unmatched."
3. "This 22 GB Stata `.dta` will not fit in memory. Rewrite `clean_admin.do` using `gtools` (`gcollapse`, `gegen`) and `frame`s instead of `preserve`/`restore`. Benchmark."

Pitfalls. Claude defaults to pandas where Polars or DuckDB would be faster — specify the engine. Stata's silent drops on `merge m:1` require `assert _merge != 1`. Indian administrative data often mixes UTF-8 and Latin-1; PIN codes lose leading zeros if stored as integers.

4.2 Econometric Analysis

Stata.

- "Run TWFE: `reghdfe log_wage treated##post X, absorb(state_id##year) cluster(state_id)`. Export with `esttab` to booktabs LaTeX. Save `tables/main.tex`."
- "Convert `analysis.do` from `xtreg, fe` to `reghdfe` and `ivreghdfe`. Verify estimates match to four decimals."
- "Coefplot of an event study: `coefplot, keep(L*.treat F*.treat) vertical yline(0)`, drop $t = -1$, shade the pre-period gray."

R / Python.

- "Re-estimate `01_main.R` with `fixest::feols`; export with `modelsummary` + `kableExtra` to booktabs LaTeX."
- "Port the Stata regression to Python via `pyfixest` (Apoorva Lal's `fixest` port). Verify cluster-robust SEs match to 10^{-6}."

- "Specification curve: 2^7 covariate combinations via `pyfixest` in joblib parallel."

The Stata hallucination problem. Mikko Rönkkö on Statalist describes Claude as "a bit less capable with Stata than with R." Documented failures include: inventing `reghdfe` options that do not exist; defaulting to plain TWFE for staggered designs (the operator must explicitly request Callaway–Sant'Anna or `did_multiplegt`); and confusing the standard-error conventions of `xtreg, fe` versus `reghdfe`. The mitigation stack is to install `dylantmoore/stata-skill` together with `sepinetam/stata-mcp`. Without the MCP, Claude can write `.do` files but cannot iterate on Stata's error output.

4.3 Replication Packages (AEA)

The eight-section AEA README structure: Overview; Data Availability Statements; Dataset List; Computational Requirements; Description of Programs; Instructions to Replicators; Table and Figure Mapping; References. Templates: `AEADataEditor/replication-template` and `AEADataEditor/docker-stata`. Recommended directory layout:

```
project/
|-- README.pdf (AEA template)
|-- data/{raw, derived}/
|-- code/{00_setup.R, 01_dataprep/, 02_analysis/, 03_appendix/}
`-- results/
```

Audit prompts:

1. "Audit my package against the AEA Data Editor checklist. For every table and figure in `paper.tex`, identify the producing line. Flag missing data citations."
2. "Set up `renv` and pin all R versions; create a Dockerfile based on `aeadataeditor/docker-stata`. Run end-to-end."
3. "Run `master.do`. Diff every output against `results/expected/`. Fail loudly on any discrepancy greater than 10^{-6}."

Pitfalls. Claude generates plausible-looking master scripts that do not run end-to-end. Always test in a clean container. Hard-coded paths sneak in — a pre-commit hook that greps for `C:\` or `/Users/` catches most.

4.4 LaTeX

- "Convert `tables/main.csv` (esttab output) into booktabs with a midrule between Panels A and B, escaped column headers, and a tablenotes block for cluster SEs."
- "Build Beamer slides from `paper/main.tex` Section 4. One result per slide, title-as-assertion (e.g., 'Treatment increased earnings 8.4%'), no bullets, `metropolis` theme."
- Tyler Ransom's BibTeX-maker prompt: "Act as a BibTeX entry maker — pass jumbled metadata, return BibTeX. Full names, periods after middle initials, Chicago title case, DOI or URL, lowercase bibkey."

LaTeX is among Claude's strongest ecosystems. The principal pitfall is occasional invention of package names; require it to enumerate every `\usepackage` it adds. Beamer overflow detection requires actually compiling, which Sant'Anna's slide-auditor agent automates.

4.5 Literature Review and PDF Handling

Claude Code reads PDFs natively, sending bytes directly to the model. This is token-expensive for typical economics papers. Panjwani's pattern is to "update the skill to use `pandoc`, `pdfgrep`, and `pdfplumber` to query documentation more efficiently." Cunningham's MixtapeTools includes a PDF-reading skill.

Sample prompts:

- "Read all PDFs in `lit/`. Extract: identification strategy, main estimator, key effect size with SE, replication-package availability. Output a sortable Markdown table."
- "For papers in `lit/did/` using staggered adoption with plain TWFE, list candidates for re-analysis with Callaway–Sant'Anna."

For bibliography management, Zotero with Better BibTeX auto-export to a committed `.bib` file is the recommended pipeline. Hallucinated citations are the classic failure mode; *Clo-Author* and *Sant'Anna* both ship dedicated citation-fidelity audit agents.

4.6 HPC and SLURM

The pattern from Mishra-Sharma's cosmological Boltzmann-solver case study (Anthropic, March 2026):

```bash
#!/bin/bash
#SBATCH --job-name=claude-agent
#SBATCH --gres=gpu:h100-32:1
#SBATCH --time=48:00:00
cd $PROJECT/my-project && source .venv/bin/activate
tmux new-session -d -s claude "claude; exec bash"
tmux wait-for claude
```

Reattach via `srun -jobid=JOBID -overlap -pty tmux attach -t claude`.

Known bugs. GitHub issue #12507: Claude Code exits on HPC interactive sessions because stdin is consumed by shell-detection logic. Issue #21026: the bash tool fails when `/run/user/` is absent. The pragmatic workaround is to run Claude Code on the login node and have it write `sbatch` jobs, or to use `dongwookim-ml/slurm-mcp` for remote submission.

USC CARC. No CARC-specific Claude Code documentation has been published. Generic patterns apply: SSH to `discovery.usc.edu`, start `tmux`, launch Claude Code, and have it write `sbatch` scripts targeting CARC partitions.

4.7 GIS and Spatial Workflows

Sample prompts:

- "Read `shapefiles/india_districts_2011.shp` with GeoPandas. Project to EPSG:7755. Spatial-join with `raw/villages.csv` (lat/lon WGS84). Assert greater than 99% match."
- "DHS data with displaced coordinates (5 km rural / 2 km urban). Buffer correctly and aggregate VIIRS nightlights via `rasterio zonal_stats`."

Pitfalls. CRS errors are silent and lethal. Force `CLAUDE.md` to require printing `df.crs` after every load.

4.8 Polyglot Projects (Stata + R + Python + LaTeX)

A master Makefile in the Gentzkow–Shapiro lineage:

```
.PHONY: all clean data analysis paper
all: paper

data/derived/panel.parquet: code/01_clean.do data/raw/*.dta
        stata -b do code/01_clean.do && python code/01b_validate.py

tables/%.tex: code/02_%.R data/derived/panel.parquet
        Rscript $<

paper/main.pdf: paper/main.tex tables/*.tex figures/*.pdf
        cd paper && latexmk -xelatex main.tex
```

The convention is one language per task: Stata for IPUMS/DHS extraction, R for econometrics, Python for scraping and spatial work, LaTeX for the paper. Inter-language exchange is Parquet only. Path-scoped `CLAUDE.md` rules are placed in each subdirectory.

5 Recommended Practices

5.1 A Composite `CLAUDE.md` Template

The following ∼120-line template is fork-ready:

```
# CLAUDE.md - [Project Name]

## Project
- Author: [Name], [Institution]
- Topic: [1-line]
- Stack: Stata 18, R 4.3 (fixest, modelsummary), Python 3.11
        (polars, pyfixest), XeLaTeX
- Master entry: ‘make all‘

## Core Principles
1. Plan-first for any task touching >2 files or >30 min
2. Replication-first: replicate paper-of-record to <1e-2 before extending
3. Verify-after: actually run code; never claim "done" without execution
4. No silent drops: every merge asserts on _merge flags

## Folder Structure
data/{raw,derived}/ # parquet only in derived/
code/{stata,R,python}/
paper/ # main.tex, refs.bib (Better BibTeX export)
tables/ figures/
results/expected/ # gold-standard for replication audit

## Conventions
- FE engine: reghdfe (Stata) | feols (R) | pyfixest (Python).
  Never plain xtreg, fe.
- Staggered DiD: Callaway-Sant’Anna or did_multiplegt; never plain TWFE.
- Cluster SE level: district_id (treatment level)
- Tables: booktabs + esttab/modelsummary; stargazer forbidden
- Random seed: 20260415 set in every master script

## Stata-specific
```

```
- ssc install reghdfe, ivreghdfe, ftools, gtools, estout, coefplot, csdid
  at top of master.do
- Use ‘frame‘ not preserve/restore for big data
- For files >5GB use gtools, NOT collapse

## Forbidden
- pdflatex (use xelatex)
- pandas for files >5GB (use polars/duckdb)
- Absolute paths
- Non-deterministic seeds (Sys.time(), runiform())
- Editing data/raw/

## Replication audit hooks
- Pre-commit: ‘make check‘ runs verify_outputs.py diffing
  tables/ vs results/expected/
- Hard fail on numbers differing by >1e-6
```

5.2 A Recommended `settings.json` for Academics

```
{
  "model": "claude-sonnet-4-6",
  "permissions": {
    "allow": ["Read","Glob","Grep","Bash(git status:*)","Bash(git diff:*)",
              "Bash(Rscript:*)","Bash(make test:*)","Bash(quarto render:*)"],
    "deny": ["Read(./.env*)","Read(./data/raw/**)","Read(./irb/**)",
              "Bash(rm -rf:*)","Bash(git push --force:*)"]
  },
  "hooks": [
    {"matcher": "Bash", "hooks": [{"type": "command",
      "command": ".claude/hooks/git-safe.sh"}]}
  ],
  "env": {"DISABLE_TELEMETRY": "1", "MAX_THINKING_TOKENS": "31999"}
}
```

5.3 Git Workflow and Worktrees

Worktrees are the highest-leverage productivity practice. As of v2.1.50, the CLI supports them natively: `claude -worktree feature-staggered-did` creates `.claude/worktrees/feature-staggered-did/` on a new branch. Manually: `git worktree add ../proj-did feature/did && cd ../proj-did && claude`. The practical ceiling is three to five parallel sessions; scope by module rather than by task. Rebase rather than merge between worktrees. Boris Cherny ships 20–30 PRs per day using this pattern.

Conventional commits are recommended; specify the convention explicitly in `CLAUDE.md`: “Use Conventional Commits. Scopes: `data`, `model`, `paper`, `slides`, `replication`.” PRs are created via the `gh` CLI. `git diff` is mandatory before approving any change. A useful adversarial pattern is to pipe the diff to a fresh-context Claude: `git diff main..HEAD | claude -p "review for econometric correctness"`.

5.4 Sensitive Data: What Not to Send

- **PII** (names, SSNs, DOBs, addresses) — never.
- **IRB-restricted human-subjects data** — protocols almost certainly forbid exfiltration.

- **FERPA-protected** student records.
- **HIPAA PHI.** Anthropic's HIPAA readiness covers `api.anthropic.com` only — *not* the Claude Code product. This is a frequent and consequential confusion.
- **Title 13 / 26** (Census, IRS) — FSRDC and Census HQ only; Claude Code is not approved inside enclaves.
- **Proprietary administrative data** (Nielsen, Visa scanner data, Indian Aadhaar-linked records) — read the data-use agreement carefully.

Account	Trains on data?	Retention
Free / Pro / Max consumer (incl. CC via consumer login)	*Yes if you do not opt out* (default flipped September 2025)	5 yr opt-in / 30 d opt-out
API (commercial)	No	30 d (7 d under new policy)
Team / Enterprise	No	Per UI
ZDR (commercial only)	No; deleted in minutes	∼0

Table 4: Anthropic data-retention by account type.

Anthropic data-retention reality. ZDR is commercial-API and Enterprise only. Pro and Max accounts *cannot* obtain ZDR. "Claude Code signed in with a personal account is still under consumer terms" — a common misconception.

Practical settings. Set `DISABLE_TELEMETRY=1` and `CLAUDE_CODE_DISABLE_FEEDBACK_SURVEY=1`. Avoid `/bug`, which bundles the session for transmission. For sensitive datasets, route through AWS Bedrock or GCP Vertex inside an institutional cloud account. The *schema-only* pattern for restricted data is: `head -0 data.csv > schema.csv`, send the schema, develop the pipeline against synthetic data, then port to the secure enclave.

Hard enforcement via PreToolUse hooks. As noted, deny rules are unreliable per multiple GitHub issues. The reliable mechanism is a hook:

```
{"hooks": [{"matcher": "Bash", "hooks": [{"type": "command",
  "command": "scripts/check-no-pii.sh"}]}]}
```

5.5 Cost Management

Default to Sonnet 4.6. Reserve Opus 4.7 for stuck debugging, identification design, and replication audits. Use Haiku 4.5 for high-volume mechanical work. The alias `-model opusplan` runs Opus during planning and Sonnet during execution. Track spend with `npx ccusage daily`. Cap headless runs with `claude -p "..." -max-budget-usd 5.00`. A three-agent Agent Team consumes approximately seven times the tokens of a single agent: a one-hour three-agent session is approximately equivalent in cost to a full single-agent day.

5.6 Common Pitfalls

1. **Stata API hallucination** — invented `reghdfe` options. Mitigate with `stata-skill` + `stata-mcp`.
2. **Methodological drift** — in the Hall audit, Claude omitted always-treated states, miscoded one county's treatment year, and omitted non-presidential elections. Replication-first protocols catch this.
3. **Auto-compaction context loss** (issue #36068) — drops user-stated intent. Mitigate by compacting at 60% proactively with explicit preservation; a `PreCompact` hook can save a snapshot.
4. **Loops on broken tests** — after two failed attempts, escalate with "ultrathink. Consider that the test may be wrong" or invoke `/rewind`.
5. **Sycophancy** — never ask yes/no. Ask "find at least three issues with this" or "critique as Referee 2."
6. **Cost runaway** — Opus + auto-accept + extended thinking + Agent Teams can exceed $100/day.
7. **Permission fatigue** leading to unsafe blanket approval. Use Auto Mode (background classifier) or sandbox mode.
8. **Hallucinated user input** (issue #10628) — Claude inserts `###Human:` mid-output. `/clear` immediately when this is observed.

5.7 Replication Audits

A reproducible workflow:

1. `git clone` the package, `cd` in, run `claude`.
2. "Read README.md and the code structure. Tell me software versions, missing data, entry point. Do not run anything yet."
3. "Set up the environment (`renv` or `conda`). Run the master script. Tee to `logs/run.log`. If it fails, diagnose."
4. "Compare each table to `paper.pdf`. Flag cells differing by more than 0.001 or sign flips."
5. Adversarial pass in a fresh session: "Critique as Referee 2. Are SE clusters appropriate? Is the robustness suspiciously narrow?"

Use Opus 4.7 for this work; the model's eleven-percentage-point recall advantage on real Anthropic PRs matters here.

5.8 Prompt Engineering for Research

- "Do not write code yet. Make a plan first." (Plan Mode is `Shift+Tab`.)
- "Interview me using `AskUserQuestion` until you have a complete spec; write to `SPEC.md`; then I will start a fresh session to execute."
- "ultrathink" for one-turn high effort; `/effort high` session-wide.
- "Prove this works. Run the tests. Diff against main."
- Anti-sycophancy: "Grill me on these changes. Do not make a PR until I pass your test."
- Counter-prompt for review: "Knowing everything you know now, scrap this and implement the elegant solution."
- Iterate two to three times — Anthropic reports outputs improve significantly with iteration.

6 Empirical Evidence on Capability and Failure Modes

6.1 Grundl (2026): The Researcher-Variation Benchmark

Grundl (2026) takes a single concrete causal question — the effect of DACA eligibility on full-time employment for Hispanic and Mexican immigrants in the U.S. — and runs it through Claude Code 100 times per task variant, across three variants. He compares the resulting distribution of estimated treatment effects to the analogous human distribution from the Huntington-Klein/Pörtner researcher-variation literature. Means and medians match across the two populations. Across-Claude-run dispersion is substantial. Across-human-researcher dispersion is two-to-twenty-seven times larger. Read straight: on tasks of this complexity, Claude Code matches a trained empirical economist in accuracy and beats one in consistency. Grundl concludes that AI systems of this class "can serve as scalable tools for producing novel empirical research, large-scale replication, and robustness analysis."

The result carries unusual weight for a skeptical economist audience. It is recent (February 2026), by a Fed economist, and on a real causal-inference task — three properties absent from most LLM benchmarks.

6.2 Straus and Hall (2026): The Replication Audit

Straus and Hall (2026) audit Claude Code's replication and extension of Thompson et al. (2020), a PNAS paper on universal vote-by-mail. The replication results are striking: Claude exactly reproduces the published headline estimates, codes 29 of 30 California counties correctly on treatment timing, and produces a new election-data collection that correlates above 0.999 with the human-collected benchmark. The extension results are clinical. Asked to push the analysis to novel cuts, Claude drifts from the prompt, misses some races, collects data and then fails to use it, documents analytic choices poorly, and in one case writes that it has restricted to eventual VCA-adopting California counties while the event-study code retains never-treated observations. No outright hallucinations or absurd errors. The lesson partitions cleanly: Claude is reliable when bounded by an existing artifact — paper, code, expected output — and unreliable on open methodological extension. The catching protocol writes itself: replicate first, extend second, verify continuously.

6.3 Xu and Yang (2026): Reproducibility at Scale

Xu and Yang (2026) run Claude Code on 92 papers (215 specifications) and reproduce them all in under four minutes per paper. The result is an existence proof: Claude Code can serve as an automated AEA Data Editor companion. Several journals are now exploring exactly this deployment.

6.4 Synthesis

The three studies converge. Claude Code performs as a competent applied econometrician on bounded tasks — replication, well-posed estimation, table generation — and outperforms human researchers in consistency. It fails on open-ended methodological extension and underperforms on Stata relative to R or Python. The operational corollaries follow directly: replicate before extending, audit across languages, and install `stata-skill` + `stata-mcp` for any Stata project.

7 Conclusion

As of April 2026, Claude Code sits closer to standard equipment than to experimental novelty in empirical economic research. Three forces moved it there: an execution loop that converts hallucinations into runtime failures; a body of economist-specific tooling — skills, MCP servers, journal-calibrated audit agents; and benchmarks showing low across-run dispersion.

Three operational recommendations follow. First, default to Sonnet 4.6 and reserve Opus 4.7 for replication audits and identification design. Second, install `dylantmoore/stata-skill` and `sepinetam/stata-mcp` on any project with a meaningful Stata footprint — the highest-leverage single configuration step. Third, treat permission deny rules as advisory and PreToolUse hooks as the load-bearing safety mechanism, especially on projects touching Indian administrative data, IRB-bound surveys, or FSRDC-adjacent workflows. Anthropic's HIPAA readiness does not extend to Claude Code; plan accordingly.

Reading List

1. Korinek, A. (2025). "AI Agents for Economic Research." NBER Working Paper 34202, September. https://www.nber.org/papers/w34202
2. Korinek, A. (2023). "Language Models and Cognitive Automation for Economic Research." NBER Working Paper 30957. https://www.nber.org/papers/w30957
3. Grundl, S. J. (2026). "Claude Code as an Empirical Economist: Like Humans but Without the Tails." SSRN 6219138, February 11. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6219138
4. Goldsmith-Pinkham, P. (2026). "Claude Code for Applied Economists," Markus' Academy mini-series (Episodes 162-1 through 162-7), March–April. https://markusacademy.substack.com/p/claude-code-for-applied-economists
5. Cunningham, S. (2025–26). MixtapeTools. https://github.com/scunning1975/MixtapeTools; "Claude Code 15: The Results Are In" (PNAS replication of Card et al. 2022). https://causalinf.substack.com/p/claude-code-15-the-results-are-in
6. Sant'Anna, P. H. C. (2026). claude-code-my-workflow. https://github.com/pedrohcgs/claude-code-my-workflow
7. Sant'Anna, H. (2026). Clo-Author. https://github.com/hugosantanna/clo-author
8. Panjwani, A. (2026). "AI Agents for Economic Research," VoxDev/CEPR webinar slides, March. https://voxdev.org/sites/default/files/2026-03/AI_Agents_for_Economic_Research.pdf
9. Blattman, C. (2026). *Claude Blattman*. https://claudeblattman.com/
10. Pepinsky, T. (2026). "Agentic AI and Social Science Research Practice," January 23. https://tompepinsky.com/2026/01/23/agentic-ai-and-social-science-research-practice/
11. Straus, G. and A. B. Hall (2026). "How Accurately Did Claude Code Replicate and Extend a Published Political Science Paper?" https://www.andrewbenjaminhall.com/Straus_Hall_Claude_Audit.pdf
12. Thompson, D. M., J. A. Wu, J. Yoder, and A. B. Hall (2020). "Universal vote-by-mail has no impact on partisan turnout or vote share." *PNAS* 117(25): 14052–14056.
13. Card, D., S. Chang, C. Becker, J. Mendelsohn, R. Voigt, L. Boustan, R. Abramitzky, and D. Jurafsky (2022). "Computational analysis of 140 years of US political speeches reveals anti-immigrant rhetoric prevalent across hostile and friendly periods." *PNAS*.
14. Xu, Y. and W. Yang (2026). "Scaling Reproducibility." https://yiqingxu.org/papers/2026_ai/AI_reproducibility.pdf

15. Cherny, B. and C. Wu (2025). Latent Space podcast, May 8. https://www.latent.space/p/claude-code
16. Mishra-Sharma, S. (2026). "Long-running Claude for scientific computing." Anthropic, March 23. https://www.anthropic.com/research/long-running-Claude
17. Anthropic (2025). "Introducing the Anthropic Economic Advisory Council," April 28. https://www.anthropic.com/news/introducing-the-anthropic-economic-advisory-council
18. Anthropic (2026). "Best Practices for Claude Code." https://code.claude.com/docs/en/best-practices
19. Anthropic (2025). "How Anthropic teams use Claude Code." https://www-cdn.anthropic.com/58284b19e702b49db9302d5b6f135ad8871e7658.pdf
20. StataCorp (2025). "Stata commands to run ChatGPT, Claude, Gemini, and Grok," October 7. https://blog.stata.com/2025/10/07/stata-commands-to-run-chatgpt-claude-gemini-and-grok/
21. Ransom, T. (2026). "How I've been using Claude for the past six months." https://tyleransom.substack.com/p/how-ive-been-using-claude-for-the
22. Willison, S. (2025). "Things we learned about LLMs in 2025." simonwillison.net, December.
23. AEA Data Editor. Replication Policy and Templates. https://aeadataeditor.github.io/

Author's note. This document was prepared as background reference material for a 30-minute USC Department of Economics workshop on Claude Code for empirical economic research, April 2026. Comments and corrections welcome at `ssharma9@usc.edu`.